Applied Numerical Methods for Civil Engineering

CGN 3405 - 0002

Week 10: Linear Algebraic Equations

Xinyu Chen

Assistant Professor

University of Central Florida

Quizzes Now!

- **Today's participation** (ungraded survey): Please check out

  "`Class Participation Quiz 22`"

  Time slot: **2:30PM – 3:00PM**

  on Canvas.

Solving Linear Systems: Comparison

Solving the linear system with different methods:

$$3x_1 + 2x_2 = 18$$
$$-x_1 + 2x_2 = 2$$

❶ Graphical method

$$x_2 = -\frac{3}{2}x_1 + 9$$
$$x_2 = \frac{1}{2}x_1 + 1$$

❷ Cramer's rule:

$$x_1 = \frac{\det(\boldsymbol{A}_1)}{\det(\boldsymbol{A})}, \qquad x_2 = \frac{\det(\boldsymbol{A}_2)}{\det(\boldsymbol{A})}$$

❸ Inverse of matrix: $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \quad \Rightarrow \quad \boldsymbol{A}^{-1}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b} \quad \Rightarrow \quad \boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$

$$\boldsymbol{A}^{-1} = \frac{\mathrm{adj}(\boldsymbol{A})}{\det(\boldsymbol{A})} \qquad \text{(adjoint matrix over determinant)}$$

❹ Gauss elimination (including back substitution)

❺ LU decomposition (including both forward and back substitutions)

Gauss-Seidel Method

An **iterative approach** to solving n simultaneous linear equations.

- **Motivation** for iterative methods: Traditional methods like Gauss elimination can become problematic for very large systems.
- **Efficiency**: Iterative methods are often faster for large, sparse systems where many coefficients are zero.

Gauss-Seidel Method

Learning objectives:

- Understand the iterative formulation for solving linear systems
- Derive the update equations for each unknown variable
- Apply the Gauss-Seidel iteration scheme
- Compute absolute relative approximate errors
- Determine convergence criteria

Linear Systems

- A system of linear equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

- A system of n linear algebraic equations:

$$\sum_{j=1}^{n} a_{ij}x_j = b_i$$

  for $i = 1, 2, \ldots, n$. We want to solve for $x_1, x_2, \ldots, x_n$.

- Matrix form $\boldsymbol{Ax} = \boldsymbol{b}$

Linear Systems

If the diagonal entries $a_{11}, a_{22}, \ldots, a_{nn}$ are nonzero, then

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3 - \cdots - a_{1n}x_n}{a_{11}}$$
$$x_2 = \frac{b_2 - a_{21}x_1 - a_{23}x_3 - \cdots - a_{2n}x_n}{a_{22}}$$
$$\vdots$$
$$x_n = \frac{b_n - a_{n1}x_1 - a_{n2}x_2 - \cdots - a_{n,n-1}x_{n-1}}{a_{nn}}$$

Thus,

$$x_1 = \frac{b_1 - \sum_{j\neq 1} a_{1j}x_j}{a_{11}} \quad x_2 = \frac{b_2 - \sum_{j\neq 2} a_{2j}x_j}{a_{22}} \quad \cdots \quad x_n = \frac{b_n - \sum_{j\neq n} a_{nj}x_j}{a_{nn}}$$

For any $x_i, i = 1, 2, \ldots, n$, we have

$$x_i = \frac{1}{a_{ii}}\left(b_i - \sum_{j\neq i} a_{ij}x_j\right) \qquad \text{(Solve each equation for one variable)}$$

Gauss-Seidel Iteration

- For a general system with right-hand side b_i:

$$
\begin{aligned}
x_i &= \frac{1}{a_{ii}}\Big(b_i - \sum_{j\neq i} a_{ij}x_j\Big) \\
&= \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j - \sum_{j>i} a_{ij}x_j\Big), \quad i = 1, 2, \ldots, n
\end{aligned}
$$

Hint:

$$
\begin{array}{ccccccccccc}
x_1 & x_2 & \cdots & x_{i-1} & \boxed{x_i} & & & & & & \text{(for all } j < i) \\
 & & & & & x_{i+1} & x_{i+2} & \cdots & x_n & & \text{(for all } j > i)
\end{array}
$$

- **Key idea**: Start with an initial point, then iteratively update each x_i using the most recent values of the other variables.
- In Gauss-Seidel iteration, we use the most recent estimates:

$$
x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\Big)
$$

with iterate k.

Convergence Criteria

- At the end of each iteration, compute the absolute relative approximate error for each variable:

$$\varepsilon_i = \left| \frac{x_i^{(k+1)} - x_i^{(k)}}{x_i^{(k+1)}} \right|$$

- Stopping condition:

$$\varepsilon_i \leq \epsilon \qquad \text{for all } i = 1, 2, \ldots, n$$

where ϵ is the pre-specified tolerance (e.g., 0.01%).

Simple 2×2 Matrices

A simple linear system:

$$4x_1 + x_2 = 9$$
$$x_1 + 3x_2 = 8$$

- Gauss-Seidel iteration:

$$x_1 = \frac{1}{4}(9 - x_2) \qquad x_2 = \frac{1}{3}(8 - x_1)$$

with initial points: $x_1^{(0)} = 0$ and $x_2^{(0)} = 0$.

Simple 2×2 Matrices

A simple linear system:

$$4x_1 + x_2 = 9$$
$$x_1 + 3x_2 = 8$$

- Gauss-Seidel iteration:

$$x_1 = \frac{1}{4}(9 - x_2) \qquad x_2 = \frac{1}{3}(8 - x_1)$$

  with initial points: $x_1^{(0)} = 0$ and $x_2^{(0)} = 0$.

- Iteratively update each variable:
  - First iteration:

$$x_1^{(1)} = \frac{1}{4}(9 - x_1^{(0)}) = \frac{9}{4} \qquad x_2^{(1)} = \frac{1}{3}(8 - x_1^{(1)}) = \frac{1}{3}(8 - \frac{9}{4}) = \frac{23}{12}$$

  - Second iteration:

$$x_1^{(2)} = \frac{1}{4} \times (9 - \frac{23}{12}) = \frac{85}{48} \approx 1.771 \qquad x_2^{(2)} = \frac{1}{3}(8 - \frac{85}{48}) = \frac{299}{144} \approx 2.076$$

- Exact solution: $x_1 = \frac{19}{11} \approx 1.727$ and $x_2 = \frac{23}{11} \approx 2.091$

Algorithm

How to solve $\boldsymbol{Ax} = \boldsymbol{b}$?

- Choose initial points $\boldsymbol{x}$ and tolerance ϵ
- Set $k = 0$
- Repeat until convergence
  - For $i = 1$ to n:
    - Compute the most recent estimate:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}\right)$$

    - Compute absolute relative approximate error:

$$\varepsilon_i = \left| \frac{x_i^{(k+1)} - x_i^{(k)}}{x_i^{(k+1)}} \right|$$

  - Check convergence: If $\varepsilon_i < \epsilon$, stop
  - Set $k = k + 1$

Solving Linear Systems

Problem statement:

$$\begin{aligned} 12x_1 + 3x_2 - 5x_3 &= 1 \\ x_1 + 5x_2 + 3x_3 &= 28 \\ 3x_1 + 7x_2 + 13x_3 &= 76 \end{aligned}$$

- Initial points $x_1 = 1, x_2 = 0, x_3 = 1$
- **Check diagonal dominance**:
  - ✓ First row: $|a_{11}| = 12 \geq |a_{12}| + |a_{13}| = 8$
  - ✓ Second row: $|a_{22}| = 5 \geq |a_{21}| + |a_{23}| = 4$
  - ✓ Third row: $|a_{33}| = 13 \geq |a_{31}| + |a_{32}| = 10$

  The inequality is strictly greater for **at least one row** $\rightarrow$ **Convergence is guaranteed**.
- Does the following linear system ensure the convergence?

$$\begin{aligned} 3x_1 + 7x_2 + 13x_3 &= 76 \\ 12x_1 + 3x_2 - 5x_3 &= 1 \\ x_1 + 5x_2 + 3x_3 &= 28 \end{aligned}$$

Solving Linear Systems

- Gauss-Seidel iteration:

$$x_1 = \frac{1}{12}(1-3x_2+5x_3) \quad x_2 = \frac{1}{5}(28-x_1-3x_3) \quad x_3 = \frac{1}{13}(76-3x_1-7x_2)$$

- Initial points $x_1^{(0)} = 1, x_2^{(0)} = 0, x_3^{(0)} = 1$
- **First iteration**:

Solving Linear Systems

- Gauss-Seidel iteration:

$$x_1 = \frac{1}{12}(1-3x_2+5x_3) \quad x_2 = \frac{1}{5}(28-x_1-3x_3) \quad x_3 = \frac{1}{13}(76-3x_1-7x_2)$$

- Initial points $x_1^{(0)} = 1, x_2^{(0)} = 0, x_3^{(0)} = 1$
- **First iteration**:
  - Update x_1:

$$x_1^{(1)} = \frac{1}{12}(1 - 3x_2^{(0)} + 5x_3^{(0)}) = \frac{1}{12}(1 - 0 + 5) = 0.5 \qquad \varepsilon_1 = 100\%$$

  - Update x_2:

$$x_2^{(1)} = \frac{1}{5}(28 - x_1^{(1)} - 3x_3^{(0)}) = \frac{1}{5}(28 - 0.5 - 3) = 4.9 \qquad \varepsilon_2 = 100\%$$

  - Update x_3:

$$x_3^{(1)} = \frac{1}{13}(76-3x_1^{(1)}-7x_2^{(1)}) = \frac{40.2}{13} = 3.0923077 \qquad \varepsilon_3 = 67.66\%$$

Solving Linear Systems

Iteration (k)	x_1	ε_1	x_2	ε_2	x_3	ε_3
1	0.5	100%	4.9	100%	3.0923	67.662%
2	0.14679	240.61%	3.7153	31.889%	3.8118	18.874%
3	0.74275	80.236%	3.1644	17.408%	3.9708	4.0064%
4	0.94675	21.546%	3.0281	4.4996%	3.9971	0.65772%
5	0.99177	4.5391%	3.0034	0.82499%	4.0001	0.07438%

- Exact solution: $x_1 = 1$, $x_2 = 3$, and $x_3 = 4$

Python Code

```python
import numpy as np

def gauss_seidel(A, b, x_init, tolerance, max_iterations=100):
    n = len(b)
    x = x_init.copy().astype(float)
    for k in range(max_iterations):
        x_old = x.copy()
        for i in range(n):
            # Compute sum for j < i (using updated x values)
            sum_j_less = np.inner(A[i, :i], x[:i])
            # Compute sum for j > i (using old x values)
            sum_j_greater = np.inner(A[i, i+1:], x_old[i+1:])
            # Update x[i]
            x[i] = (b[i] - sum_j_less - sum_j_greater) / A[i, i]
        # Calculate absolute relative approximate error
        # Avoid division by zero if x[i] is 0
        error = np.max(np.abs((x - x_old) / np.where(x != 0, x, 1)))
        if error < tolerance:
            print(f"Converged in {k+1} iterations.")
            return x
    print("Max iterations reached without convergence.")
    return x

# Example
A = np.array([[8, 2, -2], [10, 2, 4], [12, 2, 2]])
b = np.array([-2, 4, 6])
x_start = np.zeros(3) # Initial points
solution = gauss_seidel(A, b, x_start, tolerance=0.001)
print(f"Gauss-Seidel Solution: {solution}")
```

Advantage & Limitation

Advantages:

- Simple to implement
- Uses less memory than direct methods
- Good for large sparse systems
- Self-correcting (round-off not amplified)

Limitations:

- May not converge for all systems
- Requires diagonal dominance or symmetry for convergence
- Slower than direct methods for dense systems
- Convergence can be slow for ill-conditioned systems

Quick Summary

Monday's Class:

- How to solve linear systems iteratively?
- Basic idea of Gauss-Seidel method
- Algorithm & convergence criteria
- Diagonal dominance for guaranteed convergence